

PROJECT PLAN 4 — REQUIRED CORRECTIONS

Detailed rectification guide for making the AI-Driven Adaptive Multi-Objective CPU–GPU Workload Scheduling project methodologically correct and implementation-ready

Basis: PROJECT_PLAN-4.md, together with the already verified BFS/CFD implementation history and measurements available in the project conversation. This document is a correction guide, not a replacement project plan.

Overall assessment: PROJECT_PLAN-4 has a strong and substantially corrected architecture, but it is not yet final. The remaining issues are mainly internal consistency, timing semantics, ML feature separation, cost/energy definitions, hybrid execution mathematics, and reproducibility.

1. Priority Summary

Priority	Correction	Why it matters	Status
P0	Resolve BFS total-path timing formula	Prevents double-counting and invalid preferred-device labels	Must fix before labels/dataset
P0	Separate ML workload features from runtime scheduler state	Prevents a muddled prediction target and makes architecture causal	Must fix before ML
P0	Finalize CFD source timing semantics + CPU parser	CFD labels cannot be trusted until timing meaning is known	Blocking
P0	Make cost formula consistent with vCPU pricing	Otherwise CPU/GPU cost objective is dimensionally wrong	Must fix before optimizer
P1	Complete expert-feature rubrics	Makes manually derived features reproducible	Before ML
P1	Define hybrid execution time model	Current ratio alone does not define concurrent completion time	Before hybrid module
P1	Freeze objective normalization reference	Candidate-dependent scaling harms reproducibility	Before scheduler evaluation
P1	Add scheduler decision overhead metric	Scheduler itself consumes time	Before final evaluation
P2	Add ML→scheduler override analysis	Demonstrates why adaptive scheduler is more than a classifier	For evaluation/report

2. Correction 1 — BFS Total GPU Path Time

PROJECT_PLAN-4 correctly identifies the double-counting risk, but it still contains an internal contradiction: the status material describes the approximately 19.25 ms value as a validated GPU total path while also saying that the total-path formula is pending final verification.

Required change: do not call 19.25 ms a validated final value. Treat it as a current candidate/illustrative path estimate until the source-level timing semantics are finalized.

The BFS GPU phase already contains repeated synchronous GPU operations and memory copies inside the traversal loop. The separately instrumented H2D interval also includes host initialization between copies. Therefore a naive formula such as GPU phase + H2D + D2H is not automatically valid.

- Keep the raw measurements exactly as measured.
- Add a field such as **gpu_total_path_time_status** = pending_validation.
- Only create **preferred_device** after the GPU path definition is validated and comparable with the CPU timing.

- In the final report, describe ~19.25 ms only as a current estimate derived from available measurements, not as a finalized ground-truth label.

```
Correct status wording:  
BFS profiling components: verified.  
GPU total-path formula: pending validation.  
Preferred-device label: blocked until comparability is confirmed.
```

3. Correction 2 — CFD Timing Semantics and Parsers

The CFD GPU output is known to contain a value followed by the phrase “seconds per iteration”. The parser must therefore match the actual output order rather than the earlier incorrect “seconds per iteration = value” pattern.

The CPU parser must not be copied from BFS. It must be derived from the actual output of **euler3d_cpu**. The meaning of the GPU benchmark timing must also be confirmed by inspecting **euler3d.cu** before the timing is used as a comparable GPU execution metric.

- GPU parser: use the value-first pattern **r"([d.]*)s+seconds per iteration"**, subject to confirmation against the actual captured output.
- CPU parser: leave undefined until actual CPU stdout is inspected.
- Inspect the timing start/end statements and synchronization behavior in **euler3d.cu**.
- Determine whether the reported interval is kernel-only, kernel-plus-synchronization, or another benchmark-defined interval.
- Do not create CFD preferred-device labels until the CPU/GPU timing definitions are comparable.

```
CFD gating rule:  
source inspection -> timing semantics -> parser validation -> aggregation -> comparability -> label
```

4. Correction 3 — Experimental Platform Description

The platform section is broadly correct, but it should distinguish nominal hardware specifications from what the recorded Kaggle runtime actually exposed.

- CPU: report the recorded available CPU allocation (4 cores in the verified runtime) rather than only “multi-core x86-64”.
- GPU: report Tesla T4, noting 16 GB nominal VRAM and approximately 15 GB runtime-visible memory if that distinction is used.
- State that two T4 devices were visible in the runtime, while individual benchmark runs should document whether one GPU or multiple GPUs were actually used.
- Keep the explicit limitation that Kaggle is a single-node cloud-hosted heterogeneous environment, not a production multi-node HPC cluster.

5. Correction 4 — Separate ML Features from Runtime Scheduler State

This is a major conceptual correction. The preferred-device classifier and the adaptive scheduler solve related but different problems.

Intrinsic workload/task characteristics should be available before execution and should form the primary ML input. Runtime state such as GPU availability, ready-task queue length, CPU slots free, and current resource pressure belongs primarily to the scheduler.

```
Recommended architecture:  
  
Pre-execution workload features  
-> ML device suitability predictor  
-> candidate devices: CPU / GPU / Hybrid  
-> runtime scheduler  
-> dependencies  
-> transfer state  
-> CPU/GPU availability  
-> critical-path state
```

```
-> dynamic objectives
-> final assignment
```

Why: the same workload can be suitable for the GPU under one runtime state and better assigned to the CPU under another. Therefore a static preferred-device label should not be conflated with the final runtime scheduling decision.

6. Correction 5 — Workload Identity and Generalization

Features such as `workload_type` and `workload_domain` can be useful, but they can also allow the classifier to memorize workload identity rather than learn transferable performance characteristics. This matters because the plan proposes Leave-One-Workload-Out validation.

- Create a feature-set experiment without explicit workload identity.
- Create a second feature-set experiment with encoded workload/domain descriptors.
- Compare LOWO performance between the two settings.
- Do not claim broad generalization if performance depends strongly on workload identity.

7. Correction 6 — Random Forest Categorical Feature Handling

The statement that Random Forest “handles mixed feature types” should be made more precise. Standard scikit-learn `RandomForestClassifier` does not directly consume arbitrary string categories.

- Encode categorical features numerically or one-hot encode them.
- No feature scaling is required for Random Forest.
- Document the encoding pipeline and fit it only on training data when appropriate.

Correct wording:

“Random Forest handles numerical features and encoded categorical features without requiring feature scaling.”

8. Correction 7 — Complete the Expert Feature Rubrics

PROJECT_PLAN-4 provides a reproducible rubric for `estimated_parallelism` but only states that similar rubrics will be defined for memory intensity and compute intensity. Those rubrics must actually exist before model training.

- Define `estimated_parallelism` on a fixed 1–5 scale.
- Define `estimated_memory_intensity` on a fixed 1–5 scale.
- Define `estimated_compute_intensity` on a fixed 1–5 scale.
- Define objective criteria for `has_irregular_memory` and any serial-dependency flags.
- Store the rubric version with the dataset so feature assignments are reproducible.

Important: the table above is a structure to complete, not a license to assign scores subjectively. Your final rubric must contain concrete code/source-based criteria.

Feature	Score 1	Score 3	Score 5
Parallelism	Mostly sequential	Moderate parallelism	Embarrassingly parallel
Memory intensity	Compute-dominant	Balanced	Memory-dominant
Compute intensity	Low arithmetic work	Moderate	High arithmetic work

9. Correction 8 — Static Transfer Size Must Stay Pre-execution

The plan correctly states that `estimated_transfer_size_mb` must not be calculated from measured transfer duration. Make the definition operational.

- Estimate bytes crossing host/device boundaries from static input structure and source-level buffers.
- Use element counts, element sizes, and known buffer transfers where determinable.

- If a workload cannot be reliably characterized, store unavailable rather than inventing a value.
- Keep measured H2D/D2H times exclusively in post-execution telemetry.

10. Correction 9 — Transfer Ratio for BFS Must Be Deferred

The plan defines `transfer_ratio` as transfer time divided by compute time, but BFS does not yet provide a clean pure-compute interval because the instrumented GPU phase contains synchronous memory operations.

- Do not calculate a BFS `transfer_ratio` from an assumed decomposition.
- Either instrument a defensible pure-compute interval or mark the metric unavailable for BFS.
- Do not use unavailable `transfer_ratio` as a reason to fabricate a scheduler feature.

11. Correction 10 — Cost Model

The cost formula must match the unit of the pricing rate. A vCPU-hour rate is not equivalent to a whole-CPU hourly rate.

```
CPU modelled cost = execution_time_hours × allocated_vCPU_count × rate_per_vCPU_hour
GPU modelled cost = execution_time_hours × allocated_GPU_count × rate_per_GPU_hour
```

Select one documented cloud provider/configuration for the experiment and record provider, instance type, region, pricing type, source, and access date. Do not write “GCP/AWS equivalent” in the final methodology. This is a modeled cost, not Kaggle billing.

12. Correction 11 — Energy Proxy

The assumed-power-times-time approach is acceptable as a proxy, but it must never be presented as direct power measurement or actual Kaggle energy consumption.

- Label fixed 65 W CPU / 70 W T4 values, if retained, as assumed power proxies.
- Where practical, NVIDIA power sampling can be collected as an additional runtime proxy; document its limitations.
- Use a clearly defined execution interval for the energy calculation.
- If the GPU interval is only a benchmark phase rather than end-to-end path time, say so explicitly.

13. Correction 12 — Hybrid Execution Needs a Real Time Model

The current GPU partition ratio is not sufficient to define hybrid completion time. If CPU and GPU execute their partitions concurrently, the completion time is approximately governed by the slower partition plus synchronization/transfer overhead.

```
For partition ratio r on GPU:
T_CPU(r) = time for CPU portion
T_GPU(1-r) = time for GPU portion
T_hybrid(r) ≈ max(T_CPU(r), T_GPU(1-r)) + synchronization + required transfer overhead

Choose r to minimize T_hybrid(r), subject to 0 ≤ r ≤ 1.
```

Only implement this for workloads whose algorithm and data layout actually permit safe partitioning. A hybrid module should not be forced onto BFS/CFD merely to make the architecture look complete.

14. Correction 13 — Scheduler Needs a Source of Execution-Time Estimates

The classifier predicts a class; it does not produce numerical execution time. The multi-objective scorer needs candidate timing estimates.

- Phase 1: use validated Layer-2 median timings as lookup estimates for known workload/input configurations.
- Phase 2: add a regression model only if the measured dataset is sufficiently large and stable.

- For hybrid candidates, derive estimates from the partition model rather than pretending the classifier predicts throughput.

15. Correction 14 — Objective Normalization

The min-max formula is valid, but the normalization reference must be fixed for reproducibility. Do not switch between candidate-set ranges and training-dataset ranges.

```
Recommended:
Fit reference min/max values using training/profiling data.
Freeze them for test-time scheduling.

norm(x) = (x - x_min_train) / (x_max_train - x_min_train + epsilon)
```

If an objective is unavailable, mask it and renormalize the remaining active weights so that they sum to 1. Record which objectives were active for every scheduling decision.

16. Correction 15 — Dynamic Weight Controller

The dynamic-weight concept is sound, but runtime pressure definitions must be explicit. Binary `gpu_free` or `gpu_busy` state is a coarse proxy, not true queue utilization.

- Call binary state a scheduler-state pressure proxy.
- If using queue length, define it as the scheduler-managed ready/queued task count, not a claim about a production GPU queue.
- Define thresholds before evaluation.
- After applying trigger adjustments, clip weights to valid bounds and renormalize.
- Remove deadline proximity unless deadlines are explicitly added to the project.

17. Correction 16 — Dependency/DAG Definition

Keep the explicit statement that Rodinia does not provide a universal scheduler DAG. The project creates a scheduler-level DAG for controlled experiments.

- Provide at least one concrete DAG example in the implementation report.
- Define task nodes, dependency edges, and ready-task detection.
- Use longest-path calculations only as critical-path estimates unless a full scheduling/makespan proof is implemented.
- Do not describe the project-created DAG as an intrinsic Rodinia DAG unless the benchmark source actually establishes those dependencies.

18. Correction 17 — Baseline Reproducibility

All baselines must execute the same task graph, inputs, dependency constraints, and measurement protocol. Otherwise the comparison is not meaningful.

- CPU-only: all eligible tasks assigned to CPU.
- GPU-only: all GPU-supported tasks assigned to GPU.
- Rule-based: use a predeclared speedup threshold, frozen before test evaluation.
- Best-known assignment: the best measured assignment among the evaluated strategies; do not call it globally optimal.

19. Correction 18 — Add Scheduler Decision Overhead

The current evaluation treats scheduler overhead as optional. Make it a required metric because the scheduler itself performs ML inference, DAG analysis, transfer calculations, weight adaptation, and scoring.

- `scheduler_decision_time_ms`

- ML inference time if measurable
- DAG analysis time if measurable
- objective scoring time if measurable
- Total scheduling overhead per decision

20. Correction 19 — Evaluate ML Prediction vs Scheduler Override

This is a valuable analysis because it demonstrates the distinction between the ML predictor and the adaptive scheduler.

```
For each task:
ML prediction = CPU/GPU suitability
Scheduler decision = final CPU/GPU/Hybrid assignment
Measured preferred device = ground-truth label when timing is comparable

Record:
- prediction_correct
- scheduler_override
- override_reason

Example reason: "ML predicted GPU; scheduler selected CPU because transfer overhead was high and GPU pressure
was elevated."
```

21. Correction 20 — Experimental Dataset Structure

Maintain three explicit layers. Do not collapse raw repeated runs directly into the ML table.

- **Layer 1 — Raw execution:** one row per run/device/input, including failures and raw stdout/stderr metadata.
- **Layer 2 — Aggregated performance:** valid runs summarized by median, standard deviation, failure count, and timing comparability status.
- **Layer 3 — ML/scheduler:** only pre-execution features plus the ground-truth label where comparability is validated; post-execution telemetry stays separate.
- Six repetitions are repetitions, not six independent workloads for ML generalization claims.

22. Correction 21 — Validation Strategy and Dataset Size

LOWO is appropriate for testing cross-workload generalization, but the current dataset will initially be small. Report per-workload results and macro-averaged metrics, and state instability if LOWO estimates vary strongly.

- Do not randomly split repeated runs from the same workload across train/test and call that independent generalization.
- Group repetitions by workload/input configuration when constructing evaluation folds.
- Report confusion matrix, precision, recall, F1, and per-workload behavior.
- Compare against a simple baseline such as majority class or fixed-device rule where appropriate.

23. Correction 22 — Failure Handling

Preserve raw failures because they are useful engineering evidence, but do not feed failed timing values into valid performance aggregates.

- Layer 1 records return code, stdout, stderr, timeout/failure reason, and run metadata.
- Layer 2 aggregates only valid runs.
- Layer 3 uses only validated timing configurations for labels.
- Report workload failure counts separately rather than silently deleting them.

24. Correction 23 — Current Status Must Reflect What Is Actually Verified

Use component-level status rather than declaring an entire workload “verified” when one critical semantic step remains pending.

```
BFS
✓ CPU execution verified
✓ GPU execution verified
✓ repeated timing collected
✓ GPU phase semantics identified
✓ H2D/D2H instrumentation collected
✗ final GPU total-path formula validated
✗ preferred-device label finalized

CFD
✓ CPU execution verified
✓ GPU execution after compatibility rebuild verified
✓ repeated GPU timing collected
✓ GPU parser corrected
✗ CPU parser finalized
✗ source timing semantics finalized
✗ preferred-device label finalized
```

25. Correction 24 — Recommended Final Architecture

After the corrections, freeze the architecture as follows:

```
Rodinia 3.1
↓
Workload-specific profiling
↓
Raw execution data
↓
Timing validation + aggregation
↓
Leakage-free pre-execution features ■■→ Random Forest device suitability
↓
Candidate CPU / GPU / Hybrid
↓
Adaptive runtime scheduler
■■ dependencies / DAG state
■■ transfer state
■■ CPU/GPU availability
■■ critical-path proxy
■■ dynamic objective weights
↓
Multi-objective scoring
↓
Final assignment
↓
Execution
↓
Runtime telemetry
■■ ■■
weight adaptation dataset growth / retraining
```

26. Exact Corrected Implementation Order

Do not begin ML training or scheduler optimization before the timing gates are closed.

1. Inspect euler3d.cu and document CFD timing semantics.
2. Capture actual CFD CPU stdout and implement/verify the CPU parser.
3. Verify CFD GPU parser against captured output.
4. Finalize BFS GPU total-path formula without double-counting.
5. Freeze the Layer-1 per-run schema and robust record-based parser design.
6. Profile additional workloads one by one using workload-specific commands and parsers.
7. Preserve all raw successes and failures.
8. Aggregate valid runs into Layer 2 with medians, spread, and comparability flags.
9. Define and freeze the expert-feature rubrics.

- 10. Construct Layer 3 using only pre-execution features.
- 11. Generate preferred_device only for comparable workload/input configurations.
- 12. Train Random Forest with workload-aware evaluation/LOWO.
- 13. Measure prediction performance and feature importance.
- 14. Implement the rule-based baseline.
- 15. Add validated lookup-based execution-time estimates.
- 16. Implement the adaptive scheduler candidate-generation layer.
- 17. Add dependency awareness and project-created DAG.
- 18. Add transfer-aware cost calculations.
- 19. Add fixed-reference objective normalization.
- 20. Add measurable dynamic objective weighting.
- 21. Add scheduler decision-overhead measurement.
- 22. Implement hybrid execution only for one technically partitionable workload first.
- 23. Add feedback/retraining.
- 24. Run CPU-only, GPU-only, rule-based, and adaptive scheduler baselines.
- 25. Analyze ML predictions versus scheduler overrides.
- 26. Only then consider optional NSGA-II, richer regression, or additional ML models.

27. Final “Do Not Proceed Until” Checklist

- BFS total GPU path formula is source-validated and non-double-counting.
- CFD source timing semantics are understood.
- CFD CPU parser is based on actual euler3d_cpu output.
- All preferred-device labels are based on comparable timing definitions.
- ML features contain no post-execution timing/energy/cost values.
- Runtime scheduler state is separated from static workload features.
- Expert-feature rubrics are written and reproducible.
- Cloud cost provider/configuration/rates are fixed and documented.
- Energy is explicitly labeled as a proxy.
- Hybrid execution uses an explicit completion-time model.
- Objective normalization uses a fixed reference.
- Scheduler overhead is measured.
- Baseline schedules use identical workloads, inputs, and dependencies.
- LOWO/grouped evaluation is used rather than leakage-prone random repetition splits.

28. What Does Not Need to Be Redesigned

- The overall project title and research question can remain.
- Rodinia 3.1 remains the workload/benchmark source.
- The three-layer dataset architecture should remain.
- Random Forest remains a reasonable primary classifier for the current small dataset.

- The adaptive scheduler remains the central contribution.
- Dependency-aware, transfer-aware, and resource-aware scheduling remain appropriate project components.
- Weighted multi-objective optimization remains the primary optimization method; NSGA-II can stay optional.
- Feedback-driven weight adaptation and slower retraining remain appropriate.
- The Kaggle limitation and non-production-HPC wording should remain explicit.

29. Final Decision

PROJECT_PLAN-4 should be treated as a strong near-final plan, not yet as the frozen implementation specification.

The most important blockers are timing comparability (BFS and CFD), separation of ML features from runtime scheduler state, and mathematically/reproducibly defined cost, hybrid, and normalization components.

Once the P0 corrections are incorporated, the plan can be frozen and implementation can proceed in the exact order in Section 26. Do not add advanced algorithms merely to increase complexity before the measurement and dataset foundations are stable.